

Lean Proof Strategy Proposal for the Scheduling-Rules Document

Stuart Swan, Platform Architect, Siemens EDA

15 May 2026

0. Goal and scope

The goal is to formalize the document's **mathematical proof core** in Lean: trace compatibility, witness construction, cutpoint correspondence, elaboration, and one-way liveness/deadlock preservation. The Lean proof would certify the formalized theorem stack under the stated assumptions, not every informal prose statement in the document.

The document's own framing is conditional: the formal results depend on assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises.

The Lean development should therefore be organized around explicit theorem instances and assumption bundles, rather than hidden global axioms.

1. Central comparison object

The proof should be parameterized by a selected comparison instance:

structure ComparisonInstance where

Sys_ref : System

RTL_B : System

Sigma_ref : Alphabet

Sigma_post : Alphabet

Sigma_DUT : Alphabet

env : ReactiveEnvironment Sigma_post

refChoice : SysRefChoice

combNetwork? : Option CombNetwork

assumptions : AssumptionBundle

witness : WitnessBundle

contracts : ModelingContracts

Sys_ref is the chosen source reference model for the theorem instance:

- ordinary bounded-FIFO case: Sys_ref = Sys_B;
- elaborated case: Sys_ref = Sys_B^elab(E) for a well-formed elaboration package E.

This follows the updated Appendix Q direction: the formal theorems do not require a unique algorithm that constructs Sys_B^elab; instead, the chosen Sys_ref is part of the theorem instance.

2. Foundation layer: bucketed executions, not flat traces

The primary execution model should be **cycle-bucketed**, not a flat sequence of eps | tick | sigma e. The reason is that the document allows multiple same-cycle Σ -events without imposing artificial order among independent same-cycle events. The earlier review correctly noted that a flat trace would either impose a fake linearization or require an added bucket abstraction anyway.

Use:

structure State where

cycle : Nat
epsPos : Nat
store : Store

with explicit advancement rules:

-- ϵ -steps advance epsPos only.
-- Same-cycle Σ -events advance epsPos only.
-- Clock ticks advance cycle and reset epsPos.

Define buckets:

structure CycleBucket (Σ : Type) where

cycle : Nat
startState : State
epsPrefix : List EpsStep
sigmaEvents : Finset Σ
sigmaOrder : PartialOrderOn sigmaEvents
epsSuffix : List EpsStep
endState : State
endQuiescent : Prop

```
def ExecutionBucketed ( $\Sigma$  : Type) :=  
  List (CycleBucket  $\Sigma$ )
```

```
def InfiniteExecutionBucketed ( $\Sigma$  : Type) :=  
  Nat  $\rightarrow$  CycleBucket  $\Sigma$ 
```

This provides the right foundation for:

- ε -quiescent cutpoints;
- rendezvous same-cycle commits;
- R2C combinational fixed-point buckets;
- J.1 ideal induction over unordered or partially ordered same-cycle units;
- B2/B3 temporal reasoning over infinite bucketed executions.

Important separation: ExecutionBucketed is the semantic carrier for state advancement, ε -quiescence, occupancy updates, R2C fixed points, and fairness/progress reasoning. It is not itself the cross-model trace-compatibility predicate.

In particular, system-level compatibility $\tau_{\text{ref},\text{system}} \approx_{\text{sys}} \tau_{\text{post},\text{system}}$ must not be defined as equality of CycleBucket lists, equality of bucket cycle numbers, or bucket-by-bucket equality of sigmaEvents. The document's Appendix J compatibility relation is an ordered cross-model observable-compatibility predicate. It preserves the explicitly required happens-before / $\leq_J$ constraints, atomic multi-endpoint units, E1–E5 obligations, and matched value labels, while allowing independent observable units to commute or to be grouped into different cycle buckets in Sys_ref and RTL_B.

Thus:

- bucket equality may be required inside explicitly atomic units, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes;
- bucket decomposition may be used to prove local state-transition and enablement facts;
- but CompatibleSys / $\approx_{\text{sys}}$ is defined over observable units and their mandated partial order, not over identical bucket decompositions.

3. Events, operation instances, and source order

Define dynamic operation universes:

```

structure DynMsgInstance where
  proc      : Process
  channel   : Channel
  kind      : MsgKind   -- Push, Pop, PushNB, PopNB
  blocking  : Bool
  sourceLoc : SourceLoc
  instanceld : Nat

```

```

structure FrontierItem where
  proc      : Process
  itemKind  : FrontierItemKind
  sourceLoc : SourceLoc
  instanceld : Nat

```

Core source-order objects are witness-specific. They range only over dynamic operation instances that actually occur in the compared execution / witness. Untaken branches and unexecuted loop iterations are not members of this dynamic universe.

```

Qexec : Process → Set DynMsgInstance
QOmega : Process → Set DynMsgInstance
Omega : Process → Set FrontierItem

```

```

structure DynamicUniverse
  (C : ComparisonInstance)
  (P : Process)
  (W : WitnessBundle C) where
  Ω_exec : Set FrontierItem
  executed_only :
    ∀ x,
    x ∈ Ω_exec →
    ExecutedInWitness C W P x
  leP :
    {x : FrontierItem // x ∈ Ω_exec} →
    {y : FrontierItem // y ∈ Ω_exec} →
    Prop
  leP_partial_order :
    PartialOrder leP
  prefS :

```

SyncCall →

Set {x : FrontierItem // x ∈ Ω_exec}

suffS :

SyncCall →

Set {x : FrontierItem // x ∈ Ω_exec}

Thus source order $\leq_P$ is not a static total order over syntactic program statements. It is a partial order over the witness-bundle-specific dynamic frontier-item universe Ω_{exec} . The selector component $W.\text{selector}$ determines ε -modeled source choices, while $W.\text{nbMap}$ records the dynamic-instance correspondence for failed non-blocking polls that produce no Σ -event.

Non-blocking calls need explicit modeling:

- failed PushNB / PopNB polls are executed dynamic instances in Q_{exec} , but produce no Σ -event;
- successful non-blocking calls produce the corresponding committed Push_c(v) / Pop_c(v) event;
- repeated NB calls are distinct dynamic source instances even if a request signal remains physically asserted.

This matches the document's issue/commit treatment of NB calls.

4. Reactive environment and modeling contracts

4.1 Reactive environment

“Fixed stimulus” should be modeled as a causal global strategy over Σ -history, not merely as a fixed stream of per-process inputs.

structure ReactiveEnvironment (Σ : Type) where

inputAt : SigmaHistory Σ → ExternalInputs

causal :

∀ $h_1 h_2$,

SamePrefix $h_1 h_2$ →

inputAt h_1 = inputAt h_2

This directly addresses the earlier issue that reactive stimulus must be interpreted as the same global Σ -causal behavior, not a function of local process history.

4.2 Modeling contracts

Direct-input contracts and array-access contracts should be parallel top-level modeling contracts, not partly folded into RRules and partly left separate.

structure ModelingContracts (C : ComparisonInstance) where

directInputs : DirectInputContract C

arrays : ArrayAccessMappingRules C

This keeps RRules close to the document's actual R-rule set and treats direct-input and external-array obligations as environment/modeling contracts.

5. Direct-input contract and late sampling

The direct-input contract should contain only primitive environmental stability/update obligations. The fact that late sampling preserves the logical anchor should be a **derived theorem**, not a separate hypothesis.

The document says hls_direct_input permits late physical sampling because the environment holds the signal stable after reset, while hls_direct_input_sync permits controlled updates only at the designated synchronization handshake. The document also states that logical signal Read/Write events are timestamped at the E2 anchor, not at any later physical RTL sampling cycle; late sampling is ε -internal.

Define:

structure DirectInputContract (C : ComparisonInstance) where

stableAfterReset :

$\forall \text{ sig } t_1 t_2,$

DirectInput sig $\rightarrow$

AfterAllReceiversOutOfReset C sig $t_1 \rightarrow$

$t_1 \leq t_2 \rightarrow$

InputValue C.env sig $t_1 = \text{InputValue C.env sig } t_2$

syncControlledUpdateOnlyAtSync :

$\forall \text{ sig } s t,$

DirectInputSyncControlled sig s $\rightarrow$

ValueChangesAt C.env sig t $\rightarrow$

SyncHandshakeOccursAt C s t

Then prove:

theorem lateSamplingPreservesLogicalAnchor

(C : ComparisonInstance)

(hDI : DirectInputContract C)

(sig : Signal)

(anchor sampleT : Cycle)

(hWindow : DirectInputAnchorWindow C sig anchor sampleT) :

InputValue C.env sig anchor =

InputValue C.env sig sampleT

Bucket-location rule:

The logical Read(sig,val) / Write(sig,val) event belongs only to the E2 anchor bucket β_{anchor} . A physical late sample, if modeled, is an ε -step in a later bucket.

The late ε -step creates no new Σ -event. lateSamplingPreservesLogicalAnchor is used only to justify the value recorded in the logical anchor bucket.

structure DirectInputBucketWF (C : ComparisonInstance) where

read_recorded_only_at_anchor :

$\forall$ sig val anchor sampleT β ,

DirectInput sig $\rightarrow$

LogicalReadEvent sig val $\in$ BucketSigmaEvents C $\beta \rightarrow$

DirectInputAnchorWindow C sig anchor sampleT $\rightarrow$

$\beta = \text{AnchorBucket C anchor}$

write_recorded_only_at_anchor :

$\forall$ sig val anchor β ,

DirectInput sig $\rightarrow$

LogicalWriteEvent sig val $\in$ BucketSigmaEvents C $\beta \rightarrow$

$\beta = \text{AnchorBucket C anchor}$

late_sample_is_epsilon_only :

$\forall$ sig anchor sampleT,

DirectInputAnchorWindow C sig anchor sampleT $\rightarrow$

PhysicalLateSample C sig sampleT $\rightarrow$

EpsilonStepOnly C sig sampleT $\wedge$

$\neg \exists$ val β ,

PhysicalSampleEvent sig val sampleT $\in$ BucketSigmaEvents C β

late_sample_value_matches_anchor :

$\forall \text{ sig anchor sampleT},$
 $\text{DirectInputAnchorWindow C sig anchor sampleT} \rightarrow$
 $\text{InputValue C.env sig anchor} = \text{InputValue C.env sig sampleT}$

This explicitly reconciles the bucket model with direct-input late sampling:
 CompatibleP and CompatibleSys compare the logical Σ -visible Read/Write events
 at β_{anchor} , never the later physical sample micro-step.

6. Temporal logic layer for B2/B3/B5/B6

B2 and B3 should not be opaque Prop fields. They are temporal assumptions over infinite executions.

The document defines B2 as weak fairness: if an action remains continuously enabled from some cycle onward, the scheduler must eventually select it. It applies to Push, Pop, and Sync operations. It also defines B3 as the $P_{\text{push}}(c) / P_{\text{pop}}(c)$ channel-progress obligations.

Define temporal operators:

```

def AlwaysFrom
  ( $\tau$  : InfiniteExecutionBucketed  $\Sigma$ )
  ( $t$  : Nat)
  ( $P$  :  $\text{Nat} \rightarrow \text{Prop}$ ) : Prop :=
   $\forall n, t \leq n \rightarrow P\ n$ 

```

```

def EventuallyFrom
  ( $\tau$  : InfiniteExecutionBucketed  $\Sigma$ )
  ( $t$  : Nat)
  ( $P$  :  $\text{Nat} \rightarrow \text{Prop}$ ) : Prop :=
   $\exists n, t \leq n \wedge P\ n$ 

```

Concrete fairness actions:

```

inductive FairAction
| push ( $q$  : DynMsgInstance)
| pop ( $q$  : DynMsgInstance)
| sync ( $s$  : SyncCall)

```

Weak fairness:


```

def WeakFairness
  (C : ComparisonInstance)
  (τ : InfiniteExecutionBucketed C.Sigma_post) : Prop :=
  ∀ a t,
    AlwaysFrom τ t (fun n => FairActionEnabled C τ a n) →
    EventuallyFrom τ t (fun n => FairActionSelected C τ a n)

```

B3 channel-progress predicates should be defined in terms of the channel-semantics layer:

B3 channel-progress predicates should be defined in terms of persistent dynamic request instances, not per-cycle existentially chosen requests.

A blocking request is persistent from cycle t until it commits when the same dynamic operation instance q keeps its `BoundaryReq` asserted throughout that interval. For Push operations, persistence also includes payload stability while the request is outstanding:

```

def PersistentRequestFrom
  (C : ComparisonInstance)
  (τ : InfiniteExecutionBucketed C.Sigma_post)
  (q : DynMsgInstance)
  (t : Nat) : Prop :=
  Blocking q ∧
  ∀ n,
    t ≤ n →
    ¬ CommitsBeforeOrAt C τ q n →
    BoundaryReqAt C τ q n ∧
    (q.kind = MsgKind.Push →
     PayloadAt C τ q n = PayloadAt C τ q t)

```

Equivalently, one may split this into `PersistentPushRequestFrom` and `PersistentPopRequestFrom`. The single definition above is preferred here because `P_push` and `P_pop` can share the same persistence premise while Push stability is enforced only in the Push case.

```

def PushStalledFullBy
  (C : ComparisonInstance)

```

```

( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
(qPush : DynMsgInstance)
(c : Channel)
(n : Nat) : Prop :=
qPush.kind = MsgKind.Push  $\wedge$ 
qPush.channel = c  $\wedge$ 
Blocking qPush  $\wedge$ 
BoundaryReqAt C  $\tau$  qPush n  $\wedge$ 
occAt c n = capacity C c

```

```

def PopStalledEmptyBy
(C : ComparisonInstance)
( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
(qPop : DynMsgInstance)
(c : Channel)
(n : Nat) : Prop :=
qPop.kind = MsgKind.Pop  $\wedge$ 
qPop.channel = c  $\wedge$ 
Blocking qPop  $\wedge$ 
BoundaryReqAt C  $\tau$  qPop n  $\wedge$ 
occAt c n = 0

```

```

def MatchingPushRequest
(qPush : DynMsgInstance)
(c : Channel) : Prop :=
qPush.kind = MsgKind.Push  $\wedge$ 
qPush.channel = c  $\wedge$ 
Blocking qPush

```

```

def MatchingPopRequest
(qPop : DynMsgInstance)
(c : Channel) : Prop :=
qPop.kind = MsgKind.Pop  $\wedge$ 
qPop.channel = c  $\wedge$ 
Blocking qPop

```

```

def PushProgressDischargedBy
(C : ComparisonInstance)

```

```

( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
(qPush qPop : DynMsgInstance)
(n : Nat) : Prop :=
  CommitAtCycle C  $\tau$  qPush n  $\vee$ 
  CommitAtCycle C  $\tau$  qPop n  $\vee$ 
  MatchingTransferCommitOn C  $\tau$  qPush qPop n

```

```

def PopProgressDischargedBy
  (C : ComparisonInstance)
  ( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
  (qPop qPush : DynMsgInstance)
  (n : Nat) : Prop :=
    CommitAtCycle C  $\tau$  qPop n  $\vee$ 
    CommitAtCycle C  $\tau$  qPush n  $\vee$ 
    MatchingTransferCommitOn C  $\tau$  qPush qPop n

```

Then:

```

def P_push
  (C : ComparisonInstance)
  ( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
  (c : Channel) : Prop :=
     $\forall$  t qPush qPop,
      MatchingPushRequest qPush c  $\rightarrow$ 
      MatchingPopRequest qPop c  $\rightarrow$ 
      PersistentRequestFrom C  $\tau$  qPush t  $\rightarrow$ 
      PersistentRequestFrom C  $\tau$  qPop t  $\rightarrow$ 
      AlwaysFrom  $\tau$  t (fun n =>
        PushStalledFullBy C  $\tau$  qPush c n)  $\rightarrow$ 
      EventuallyFrom  $\tau$  t (fun n =>
        PushProgressDischargedBy C  $\tau$  qPush qPop n)

```

```

def P_pop
  (C : ComparisonInstance)
  ( $\tau$  : InfiniteExecutionBucketed C.Sigma_post)
  (c : Channel) : Prop :=
     $\forall$  t qPop qPush,

```

MatchingPopRequest qPop c →
 MatchingPushRequest qPush c →
 PersistentRequestFrom C τ qPop t →
 PersistentRequestFrom C τ qPush t →
 AlwaysFrom τ t (fun n =>
 PopStalledEmptyBy C τ qPop c n) →
 EventuallyFrom τ t (fun n =>
 PopProgressDischargedBy C τ qPop qPush n)

This matches the document's conditional B3 reading: B3 forces progress only when both endpoints continuously request the matching transfer; a producer stalled on a full channel is not automatically a B3 violation if the consumer is not yet requesting the matching pop.

Bundle B-rules:

structure BRules (C : ComparisonInstance) where
 B1_determinism : DeterministicSigmaTrace C

B2_weakFairness :

∀ τ, InfinitePostExecution C τ →
 WeakFairness C τ

B3_channelProgress :

∀ τ c, InfinitePostExecution C τ →
 P_push C τ c ∧ P_pop C τ c

B4_finiteEpsilonDepth : FiniteEpsilonDepth C

B5_quiescentNormalization : QuiescentNormalization C

B6_idleStutterTimeDivergence : IdleStutterTimeDivergence C

B2/B3 remain assumptions, but now they are structured temporal assumptions usable by K proofs.

7. Channel semantics and E4

Define channel state over the selected Sys_ref boundary.

The primitive abstract occupancy is cycle-boundary, non-fall-through occupancy. It is indexed by the cycle, not by the full intra-cycle state:

$\text{capacity} : \text{C.Sys_ref.Channel} \rightarrow \text{Nat}$
 $\text{occAt} : \text{C.Sys_ref.Channel} \rightarrow \text{Cycle} \rightarrow \text{Nat}$

For convenience, one may define a state-indexed abbreviation only by projecting the state to its cycle:

$\text{occAtState} : \text{C.Sys_ref.Channel} \rightarrow \text{State} \rightarrow \text{Nat}$
 $\text{occAtState } c \ \sigma := \text{occAt } c \ \sigma.\text{cycle}$

This abbreviation is not a second occupancy notion. In particular, occupancy is invariant under ε -steps and under same-cycle Σ -events:

$\text{occ_invariant_within_cycle} :$
 $\forall c \ \sigma_1 \ \sigma_2,$
 $\sigma_1.\text{cycle} = \sigma_2.\text{cycle} \rightarrow$
 $\text{occAtState } c \ \sigma_1 = \text{occAtState } c \ \sigma_2$

$\text{BoundaryReq} : \text{DynMsgInstance} \rightarrow \text{State} \rightarrow \text{Prop}$
 $\text{ChannelEnabled} : \text{DynMsgInstance} \rightarrow \text{State} \rightarrow \text{Prop}$
 $\text{ChannelDisabled } q \ \sigma := \text{BoundaryReq } q \ \sigma \wedge \neg \text{ChannelEnabled } q \ \sigma$

ChannelEnabled is allowed to inspect the settled boundary-request state σ , but its capacity/occupancy component must use $\text{occAt } q.\text{channel } \sigma.\text{cycle}$. Thus E4 enablement is evaluated against the beginning-of-cycle abstract occupancy, with no same-cycle fall-through through ε -settling or same-cycle Σ -events.

Capacity cases:

inductive CapacityKind
| rendezvous -- $B(c) = 0$
| buffered -- $B(c) > 0$

Expected lemmas:

$\text{BufferedPushEnabled}$
 $\text{BufferedPopEnabled}$
 RendezvousEnabled
 $\text{FIFOOrderPreservation}$
 NoDropNoDuplicate

OccupancyUpdatePush
OccupancyUpdatePop

E4 should be parameterized by Sys_ref, so in elaborated cases it ranges over explicit elaborated channels, not merely outer channels.

8. Appendix Q elaboration interface

Appendix Q should be introduced early. In the updated document, elaboration is not a late proof afterthought: in elaborated cases, Sys_ref = Sys_B^elab(E), and all references to B, occ, BoundaryReq, ChannelEnabled/Disabled, Front_P, and WFG are interpreted over the explicit channels of the elaborated model.

As in the ordinary E4 semantics, elaborated-channel occupancy is cycle-boundary, non-fall-through occupancy. Thus occ_E(d,t) and A_E(c,t) are indexed by cycle t. State-indexed forms such as occ_E(d,σ) or A_E(c,σ), when used informally at ε-quiescent cutpoints, are only abbreviations for occ_E(d,σ.cycle) and A_E(c,σ.cycle).

Chan_outer should be a parameter fixed by the chosen outer Sys_B, not a field chosen internally by the elaboration package.

structure ElaborationPackage
(Sys_B : System)
(Chan_outer : Type)
(Σ_DUT : Type) where

Sys_elab : System

Chan_elab : Type
finite_chan_elab : Fintype Chan_elab

B_E : Chan_elab → Nat
occ_E : Chan_elab → Cycle → Nat

Pi_elab : Chan_elab → Option Chan_outer
piSigma : Trace Sys_elab.Sigma → Trace Σ_DUT
A_E : Chan_outer → Cycle → Nat

```

outer_trace_preservation : Prop
occupancy_preservation : Prop
internal_refinement : Prop
no_hidden_cross_channel_disablement : Prop
wfg_visibility : Prop
combinational_wellformedness : Prop

```

For convenience, the elaboration package may also define state-indexed abbreviations by projecting the state to its cycle:

```

occ_E_atState : Chan_elab → State → Nat
occ_E_atState d σ := occ_E d σ.cycle

```

```

A_E_atState : Chan_outer → State → Nat
A_E_atState c σ := A_E c σ.cycle

```

These abbreviations are not separate occupancy or accounting notions. They are invariant under ε -steps and same-cycle Σ -events:

```

occ_E_invariant_within_cycle :
  ∀ d σ1 σ2,
  σ1.cycle = σ2.cycle →
  occ_E_atState d σ1 = occ_E_atState d σ2

```

```

A_E_invariant_within_cycle :
  ∀ c σ1 σ2,
  σ1.cycle = σ2.cycle →
  A_E_atState c σ1 = A_E_atState c σ2

```

Reference choice:

```

inductive ReferenceKind
| ordinarySysB
| elaborated

```

```

structure SysRefChoice where
  outerSysB : System
  Chan_outer : Type
  kind : ReferenceKind

```

elab? : Option (ElaborationPackage outerSysB Chan_outer Σ _DUT)
Sys_ref : System
sys_ref_matches_kind : Prop

For ordinarySysB, Sys_ref = outerSysB.

For elaborated, Sys_ref = E.Sys_elab.

This also handles sync-boundary epoch gates: if proof-relevant withholding exists, it must be represented by explicit gate channels in Sys_B^elab(E), not by hidden gating on an otherwise E4-enabled outer channel. The earlier review correctly flagged this as a point that must be explicit in the Lean strategy.

9. R-rules, E-rules, and modeling contracts

The R-rule bundle should include only the document's actual R-rules plus an optional Lean-side array-rule pointer if desired. It should **not** include a fictional R5_syncBoundaryNoCrossing; sync-boundary crossing is E5. The document explicitly labels "Messages cannot cross their immediate synchronization boundary" as E5.

structure RRules (C : ComparisonInstance) where
 R1_syncOrder : Prop
 R2_signalAnchorSequential : Prop
 R2C_combinationalFixedPoint : Prop
 R3_syncSemantics : Prop
 R4_messageIssueCommitOrder : Prop

E-rules:

structure ERules (C : ComparisonInstance) where
 E1_sync_order_preservation : Prop
 E2_signal_anchor_preservation : Prop
 E3_message_order_preservation : Prop
 E4_channel_capacity_semantics : Prop
 E5_sync_boundary_preservation : Prop

Array and direct-input conditions live in:

structure ModelingContracts (C : ComparisonInstance) where
 directInputs : DirectInputContract C
 arrays : ArrayAccessMappingRules C

This avoids treating environment/design-side contracts inconsistently.

10. Array access mapping

External arrays are not merely internal Store. The document states that external arrays are mapped onto IO operations by an array access mapping layer, and that array accesses before a synchronization call must commit no later than the sync commit, while accesses after the sync must commit strictly after it. It also permits transformations such as caching, merging, splitting, and reordering when allowed by the mapping layer.

Define:

inductive MemoryEvent

| read (array : ArrayId) (addr : Addr) (value : Value)

| write (array : ArrayId) (addr : Addr) (value : Value)

structure ArrayAccessMapping where

sourceArrayAccess : SourceArrayAccess → List CoreIOEvent

commitTime : SourceArrayAccess → Cycle

fixedLatency : SourceArrayAccess → Nat

conflictFreeReorderAllowed :

SourceArrayAccess → SourceArrayAccess → Prop

transformedEquivalent :

SourceArrayAccess → List SourceArrayAccess → Prop

Rules:

structure ArrayAccessMappingRules (C : ComparisonInstance) where

beforeSyncCommitsNoLater :

∀ a s,

a ∈ prefS s →

ArrayCommitTime a ≤ SyncCommitTime s

afterSyncCommitsAfter :

∀ a s,

a ∈ suffS s →

SyncCommitTime s < ArrayCommitTime a

issueConsistentWithFixedLatency :

$\forall a,$
ArrayCommitTime a =
ArrayIssueTime a + ArrayFixedLatency a

transformedOpsRespectMapping :

$\forall a$ transformed,
TransformedFrom a transformed $\rightarrow$
MappingEquivalent a transformed

conflictFreeReorderingOnly :

$\forall a_1 a_2,$
Reordered a₁ a₂ $\rightarrow$
conflictFreeReorderAllowed a₁ a₂

For early milestones, it is acceptable to assume:

AllExternalArrayAccessesAlreadyMappedToCoreIO C

but the full proof strategy should include array mapping.

11. R2C combinational fixed point

ComparisonInstance now has:

combNetwork? : Option CombNetwork

R2C should use that field when combinational processes are in scope. The document says combinational Read/Write events occur in the observable bucket corresponding to the cycle's ε -quiescent combinational fixed point, and that combinational signal IO is well formed only when the relevant network is deterministic and reaches a unique ε -fixed point.

Define:

structure CombNetwork where
combStep : Store $\rightarrow$ Store $\rightarrow$ Prop
acyclic : Prop
deterministic : Prop

acyclic and deterministic are proof obligations supplied by the network instance. They are not automatically checked unless the network representation is made concrete enough to prove them.

```
def CombQuiescent (N : CombNetwork) (σ : State) : Prop :=
  ¬ ∃ σ', N.combStep σ.store σ'.store
```

```
def CombFixedPointAtCycle
  (N : CombNetwork)
  (t : Cycle)
  (μ : Store) : Prop :=
  ReachesByCombSteps N t μ ∧
  IsFixedPoint N μ
```

Well-formedness:

```
structure R2C_WF (C : ComparisonInstance) where
  network_present :
    C.combNetwork?.isSome
```

```
uniqueFixedPoint :
  ∀ t N,
    C.combNetwork? = some N →
    ∃! μ, CombFixedPointAtCycle N t μ
```

```
sigmaReadsAtFixedPoint :
  ∀ e t μ N,
    C.combNetwork? = some N →
    CombReadWriteEvent e →
    e ∈ BucketSigmaEvents C t →
    CombFixedPointAtCycle N t μ →
    EventReadsWritesStore e μ
```

If combinational processes are out of scope for an early milestone, use:

SequentialOnly C

and postpone R2C_WF.

12. Issue/Commit well-formedness

Issue and Commit should be relational plus existence/uniqueness, not necessarily computable total functions.

IssueAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Prop

CommitAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Payload → Prop

Well-formedness:

structure IssueCommitWF (C : ComparisonInstance) where

issue_exists :

$\forall \tau q,$
IssuedInTrace C $\tau q \rightarrow$
 $\exists t, \text{IssueAt } \tau q t$

issue_unique :

$\forall \tau q t_1 t_2,$
IssueAt $\tau q t_1 \rightarrow$
IssueAt $\tau q t_2 \rightarrow$
 $t_1 = t_2$

commit_exists_unique :

$\forall \tau q,$
CommitsInTrace C $\tau q \rightarrow$
 $\exists! tp : \text{Cycle} \times \text{Payload},$
CommitAt $\tau q tp.1 tp.2$

boundary_request_soundness : Prop

stable_attribution : Prop

back_to_back_issue : Prop

sync_boundary_discipline : Prop

nb_dynamic_instance_discipline : Prop

Derived timestamp:

noncomputable def IssueTime

(hWF : IssueCommitWF C)

(h : IssuedInTrace C τq) : Cycle :=

Classical.choose (hWF.issue_exists $\tau q h$)

This preserves Appendix C's use of Issue(q) as an auxiliary timestamp while avoiding an unnecessary computability requirement.

13. Automatic flush and sync-boundary policy

Define:

Pending $P \sigma$

Front $P \sigma$

NextFront $P \sigma$

BlockingMsgNextFront $P \sigma$

Done $P \sigma$

Remain $P \sigma$

Automatic flush must be stated over the same dynamic universe and source-order objects used by Appendix I/J/K. In particular, it must not be an opaque Prop bundle.

def FrontierBlocked

(C : ComparisonInstance)

(P : Process)

(σ : State) : Prop :=

Front C P $\sigma \neq \emptyset \wedge$

$\forall x,$

$x \in \text{Front C P } \sigma \rightarrow$

BlockingMessageFrontierItem x $\wedge$

BoundaryReq C x $\sigma \wedge$

ChannelDisabled C x σ

def LaterThanBlockedFrontier

(C : ComparisonInstance)

(P : Process)

(σ : State)

(y : FrontierItem) : Prop :=

$\exists x,$

$x \in \text{Front C P } \sigma \wedge$

SourceOrderLt C P x y

def FrontierRemainsBlockedOver

(C : ComparisonInstance)

(P : Process)

$(\tau : \text{Trace } C.\text{RTL_B})$
 $(t_0 \ t : \text{Cycle}) : \text{Prop} :=$
 $\forall n \ \sigma n,$
 $t_0 \leq n \rightarrow$
 $n \leq t \rightarrow$
 $\text{StateAtCycle } \tau \ n \ \sigma n \rightarrow$
 $\text{FrontierBlocked } C \ P \ \sigma n$

$\text{def NoNewLaterWorkWhileBlocked}$
 $(C : \text{ComparisonInstance})$
 $(P : \text{Process})$
 $(\tau : \text{Trace } C.\text{RTL_B}) : \text{Prop} :=$
 $\forall t_0 \ t \ \sigma_0 \ y,$
 $\text{StateAtCycle } \tau \ t_0 \ \sigma_0 \rightarrow$
 $\text{FrontierBlocked } C \ P \ \sigma_0 \rightarrow$
 $\text{FrontierRemainsBlockedOver } C \ P \ \tau \ t_0 \ t \rightarrow$
 $\text{LaterThanBlockedFrontier } C \ P \ \sigma_0 \ y \rightarrow$
 $\neg \text{IssueAt } \tau \ y \ t$

$\text{def NoWithdrawalWhileBlocked}$
 $(C : \text{ComparisonInstance})$
 $(P : \text{Process})$
 $(\tau : \text{Trace } C.\text{RTL_B}) : \text{Prop} :=$
 $\forall t_0 \ t \ \sigma_0 \ \sigma t \ x,$
 $\text{StateAtCycle } \tau \ t_0 \ \sigma_0 \rightarrow$
 $\text{StateAtCycle } \tau \ t \ \sigma t \rightarrow$
 $t_0 \leq t \rightarrow$
 $x \in \text{Front } C \ P \ \sigma_0 \rightarrow$
 $\text{BoundaryReq } C \ x \ \sigma_0 \rightarrow$
 $\neg \text{CommitsBetween } \tau \ x \ t_0 \ t \rightarrow$
 $\text{BoundaryReq } C \ x \ \sigma t \wedge$
 $(\text{IsPushFrontierItem } x \rightarrow$
 $\text{PayloadAtState } C \ x \ \sigma t = \text{PayloadAtState } C \ x \ \sigma_0)$

$\text{def FrontierMinimality}$
 $(C : \text{ComparisonInstance})$
 $(P : \text{Process})$
 $(\tau : \text{Trace } C.\text{RTL_B}) : \text{Prop} :=$

```

 $\forall t \sigma x y,$ 
StateAtCycle  $\tau t \sigma \rightarrow$ 
 $x \in \text{Front } C \ P \ \sigma \rightarrow$ 
 $y \in \text{Remain } C \ P \ \sigma \rightarrow$ 
SourceOrderLt  $C \ P \ y \ x \rightarrow$ 
False

```

The next two predicates are the local automatic-flush instances of the $K\varepsilon / A11$ WFG-inertness obligation. They are stated here because DrainOnlyDownstreamProgress uses them directly; Appendix K later consumes the same predicates as part of the global deadlock-preservation argument.

```

def WFGInertForProcess
(C : ComparisonInstance)
(P : Process)
( $\sigma_0 \ \sigma t$  : State) : Prop :=
 $\forall Q,$ 
WFGEdge  $C \ .\text{post } \sigma_0 \ P \ Q \leftrightarrow \text{WFGEdge } C \ .\text{post } \sigma t \ P \ Q$ 

```

```

def NoNewBlockingFrontierCause
(C : ComparisonInstance)
(P : Process)
( $\sigma_0 \ \sigma t$  : State) : Prop :=
 $\forall x,$ 
 $x \in \text{Front } C \ .\text{post } P \ \sigma t \rightarrow$ 
BlockingMessageFrontierItem  $x \rightarrow$ 
ChannelDisabled  $C \ .\text{post } x \ \sigma t \rightarrow$ 
 $\exists y,$ 
 $y \in \text{Front } C \ .\text{post } P \ \sigma_0 \wedge$ 
BlockingMessageFrontierItem  $y \wedge$ 
SameBlockingCause  $C \ P \ y \ x$ 

```

```

def DrainOnlyDownstreamProgress
(C : ComparisonInstance)
(P : Process)
( $\tau$  : Trace  $C \ .\text{RTL\_B}$ ) : Prop :=
 $\forall t_0 \ t \ \sigma_0 \ \sigma t,$ 
StateAtCycle  $\tau \ t_0 \ \sigma_0 \rightarrow$ 

```

StateAtCycle $\tau \ t \ \sigma \rightarrow$
 $t_0 \leq t \rightarrow$
 FrontierBlocked $C \ P \ \sigma_0 \rightarrow$
 FrontierRemainsBlockedOver $C \ P \ \tau \ t_0 \ t \rightarrow$
 InternalOrEpsilonOnlyBetween $\tau \ t_0 \ t \rightarrow$
 WFGInertForProcess $C \ P \ \sigma_0 \ \sigma \wedge$
 NoNewBlockingFrontierCause $C \ P \ \sigma_0 \ \sigma$

structure AutomaticFlush
 (C : ComparisonInstance)
 (P : Process)
 (τ : Trace C.RTL_B) where
 block_point :
 $\forall \ t \ \sigma,$
 StateAtCycle $\tau \ t \ \sigma \rightarrow$
 FrontierBlocked $C \ P \ \sigma \rightarrow$
 FlushBlockedPoint $C \ P \ \tau \ t \ \sigma$

no_new_later_work :
 NoNewLaterWorkWhileBlocked $C \ P \ \tau$

no_withdrawal :
 NoWithdrawalWhileBlocked $C \ P \ \tau$

frontier_minimality :
 FrontierMinimality $C \ P \ \tau$

drain_only_downstream_progress :
 DrainOnlyDownstreamProgress $C \ P \ \tau$

sync_boundary_policy :
 SyncBoundaryPolicy $C \ P \ \tau$

The sync-boundary policy must explicitly split the ordinary and elaborated cases:

inductive SyncBoundaryPolicy
 (C : ComparisonInstance)
 (P : Process)
 (τ : Trace C.RTL_B)

| no_withholding_in_ordinary_SysB :

C.refChoice.kind = ReferenceKind.ordinarySysB →
NoProofRelevantSyncBoundaryWithholding C P τ →
SyncBoundaryPolicy C P τ

| explicit_gate_in_elab :

$\forall E$ gateChannel,
C.refChoice.kind = ReferenceKind.elaborated →
GateChannelOf E gateChannel →
OrdinaryChannelDisabledAtGate C P τ gateChannel →
SyncBoundaryPolicy C P τ

In the ordinary Sys_B case, the sync-boundary clause does **not** add hidden gating. It asserts that no proof-relevant sync-boundary withholding is present. If such withholding is needed, the theorem instance must use Sys_B^{elab}(E).

14. Witness selector and Appendix L

With the assumed renumbering:

- Definition L.1 = witness selector / snooping;
- Lemma L.2 = snooping establishes WC;
- Corollary L.3 = NB-CF identity witness.

The uploaded document's witness selector is a partial strategy over ε -quiescent source cutpoints and ε -modeled choices whose outcomes may affect later Σ -visible behavior; it must be causally available from the matched RTL prefix and global Σ -causal history.

Avoid circularity by defining it over the partial construction so far:

structure WitnessSelector (C : ComparisonInstance) where

choose :

($\tau_{\text{post_prefix}}$: Prefix C.RTL_B) →

($\tau_{\text{ref_so_far}}$: Prefix C.Sys_ref) →

(cp : ChoicePoint $\tau_{\text{ref_so_far}}$) →

Option ChoiceOutcome

causal :

$\forall \tau_p \tau_r \text{ cp out,}$

choose $\tau_p \tau_r \text{cp} = \text{some out} \rightarrow$
 DependsOnlyOnAvailablePrefix $\tau_p \tau_r \text{cp out}$
 consistent_with_post_prefix :
 $\forall \tau_p \tau_r \text{cp out},$
 choose $\tau_p \tau_r \text{cp} = \text{some out} \rightarrow$
 ConsistentWithRTLPrefix $\tau_p \tau_r \text{cp out}$

Non-blocking witness mapping.

Failed PushNB / PopNB polls are executed dynamic instances in Q_{exec} but produce no Σ -event. Therefore the Post $\rightarrow$ Ref construction cannot reconstruct them from the Σ -trace alone. The witness bundle must carry an explicit partial dynamic-instance correspondence for such ε -modeled NB instances.

structure NBInstanceMap (C : ComparisonInstance) where
 $\mu_Q :$
 DynMsgInstance $\rightarrow$ Option DynMsgInstance

maps_only_executed_nb :
 $\forall q_{\text{post}} q_{\text{ref}},$
 $\mu_Q q_{\text{post}} = \text{some } q_{\text{ref}} \rightarrow$
 ExecutedInPost C $q_{\text{post}} \wedge$
 ExecutedInRef C $q_{\text{ref}} \wedge$
 IsNB $q_{\text{post}} \wedge$ IsNB q_{ref}

preserves_owner_and_interface :
 $\forall q_{\text{post}} q_{\text{ref}},$
 $\mu_Q q_{\text{post}} = \text{some } q_{\text{ref}} \rightarrow$
 $q_{\text{post}}.\text{proc} = q_{\text{ref}}.\text{proc} \wedge$
 $q_{\text{post}}.\text{channel} = q_{\text{ref}}.\text{channel} \wedge$
 $q_{\text{post}}.\text{kind} = q_{\text{ref}}.\text{kind}$

preserves_nb_outcome :
 $\forall q_{\text{post}} q_{\text{ref}},$
 $\mu_Q q_{\text{post}} = \text{some } q_{\text{ref}} \rightarrow$
 NBOutcomePost C $q_{\text{post}} = \text{NBOutcomeRef C } q_{\text{ref}}$

covers_control_relevant_failed_polls :
 $\forall q_{\text{post}},$

ExecutedInPost C q_post →
 IsFailedNB q_post →
 AffectsLaterSigmaOrFrontier C q_post →
 $\exists q_ref, \mu_Q q_post = \text{some } q_ref$

source_order_position_preserved :
 $\forall q_post q_ref,$
 $\mu_Q q_post = \text{some } q_ref \rightarrow$
 SourceOrderKeyOf C q_post = SourceOrderKeyOf C q_ref

respects_source_order_slice :
 $\forall P q_1_post q_2_post q_1_ref q_2_ref,$
 $\mu_Q q_1_post = \text{some } q_1_ref \rightarrow$
 $\mu_Q q_2_post = \text{some } q_2_ref \rightarrow$
 SourceOrderLe C P (SourceOrderKeyOf C q_1_post) (SourceOrderKeyOf C q_2_post) ↔
 SourceOrderLe C P (SourceOrderKeyOf C q_1_ref) (SourceOrderKeyOf C q_2_ref)

structure WitnessBundle (C : ComparisonInstance) where
 selector : WitnessSelector C
 nbMap : NBInstanceMap C

Witness compatibility:
 def WC
 (C : ComparisonInstance)
 (W : WitnessBundle C) : Prop :=
 (∀ constructionPrefix,
 WitnessChoicesRespectMatchedPrefix C W.selector constructionPrefix) ∧
 WitnessNBMapRespectsMatchedPrefix C W.nbMap ∧
 WitnessChoicesAndNBMapAgree C W.selector W.nbMap

Lemma L.2:
 theorem L2_snooping_establishes_WC
 (C : ComparisonInstance)
 (W : WitnessBundle C)
 (hSnoop : SnoopingWrapperAssumptions C W) :
 WC C W

IdentityOrArbitraryNBWitnessBundle C is the witness bundle used in the NB-CF

case. Its selector uses identity choices for ε -modeled non-blocking outcomes that are fixed by the matched prefix, and arbitrary choices for non-blocking outcomes that NB-CF proves cannot affect later Σ -visible control flow or NextFront behavior. Its nbMap maps matched executed NB instances to the corresponding same dynamic source-call instance when such a match is relevant, and is arbitrary on failed NB instances that NB-CF proves irrelevant.

```
def IdentityOrArbitraryNBWitnessBundle
(C : ComparisonInstance) : WitnessBundle C :=
{
  selector := IdentityOrArbitraryNBSelector C,
  nbMap := IdentityOrArbitraryNBMap C
}
```

Corollary L.3:

theorem L3_nb_cf_identity_witness

(C : ComparisonInstance)

(P : Process)

(hNBCF : NBCF C P) :

WC C (IdentityOrArbitraryNBWitnessBundle C)

15. Concrete ObsSigma and deterministic replay

Do not define ObsSigma as an informal least fixed point. Define it as a concrete record of the source-state slices required for replay.

```
structure ObsSigma
(C : ComparisonInstance)
(W : WitnessBundle C)
(P : Process)
( $\sigma$  : State) where
  control_pos : ControlPos P
  relevant_vars : RelevantVarSlice C P  $\sigma$ 
  selected_eps_outcomes : SelectedOutcomeSlice C W.selector P  $\sigma$ 
  nb_witness_slice : NBWitnessSlice C W.nbMap P  $\sigma$ 
  pending_slice : PendingSlice C P  $\sigma$ 
  front_slice : FrontSlice C P  $\sigma$ 
  nextfront_slice : NextFrontSlice C P  $\sigma$ 
```

boundary_req_slice : BoundaryReqSlice C P σ
 channel_fact_slice : ChannelFactSlice C P σ
 signal_anchor_slice : SignalAnchorSlice C P σ
 array_mapping_slice : ArrayMappingSlice C P σ
 direct_input_slice : DirectInputSlice C P σ

Then:

def SigmaRelevantEq C W P $\sigma_1 \sigma_2$:=
 ObsSigma C W P σ_1 = ObsSigma C W P σ_2

Replay theorem:

theorem IDR_deterministic_replay
 (C : ComparisonInstance)
 (W : WitnessBundle C)
 (hB : BRules C)
 (hR : RRules C)
 (hE : ERules C)
 (hIC : IssueCommitWF C)
 (hContracts : ModelingContracts C)
 (hWC : WC C W)
 (P : Process)
 ($\sigma_1 \sigma_2$: State)
 (hObs : SigmaRelevantEq C W P $\sigma_1 \sigma_2$) :
 SameNextSigmaRelevantBehavior C W P $\sigma_1 \sigma_2$

This same ObsSigma object should be reused by Lemma S1 and Corollary J.1.

16. Appendix I per-process construction

Bundle the assumptions:

structure IConstructionAssumptions
 (C : ComparisonInstance)
 (W : WitnessBundle C) where
 wellformed : C.WellFormed
 bRules : BRules C
 rRules : RRules C
 eRules : ERules C

issueCommitWF : IssueCommitWF C
implementation : ImplementationAssumptions C
witnessCompatible : WC C W
contracts : ModelingContracts C

Main theorems:

theorem I1_bounded_epsilon_closure
(C : ComparisonInstance)
(hB4 : C.assumptions.B.B4_finiteEpsilonDepth)
(hB5 : C.assumptions.B.B5_quiescentNormalization) :
...

theorem I2_channel_progress_instance
(C : ComparisonInstance)
(hB3 : C.assumptions.B.B3_channelProgress) :
...

theorem I3_finite_extension_step
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I4_finite_prefix_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

theorem I5_infinite_limit_construction
(C : ComparisonInstance)
(W : WitnessBundle C)
(hI : IConstructionAssumptions C W) :
...

I3 should use IssueAt plus derived IssueTime, not an unconstrained guessed issue cycle.

17. Appendix J system-level correspondence

Define execution scopes:

```
def Exec_post (C : ComparisonInstance) (τ : Prefix C.RTL_B) : Prop :=  
  ∃ τ∞,  
    Extends τ∞ τ ∧  
    InfinitePostExecution C τ∞ ∧  
    FairProgressSatisfying C τ∞
```

```
def Exec_ref (C : ComparisonInstance) (τ : Prefix C.Sys_ref) : Prop :=  
  ∃ τ∞,  
    Extends τ∞ τ ∧  
    RefAdmissible C τ∞
```

Directed compatibility:

Do not define CompatibleP or CompatibleSys as equality of bucketed executions.

First define the observable units extracted from a bucketed execution. An observable unit may be:

- a single ordinary Σ -event;
- an explicitly atomic multi-endpoint unit, such as a rendezvous Push/Pop pair or a SyncChannel/ac_sync pair;
- a same-cycle finite set of independent Σ -events equipped with the partial order actually mandated by E1–E5.

```
structure ObservableUnit (Σ : Type) where  
  events : Finset Σ  
  order : PartialOrderOn events  
  atomic : Prop
```

ObservableUnitsOf extracts Σ -visible events from the bucketed execution and groups them into comparison units as follows:

- an ordinary committed message, signal, or synchronization event may form a singleton observable unit;
- explicitly atomic multi-endpoint events, such as rendezvous Push/Pop pairs and SyncChannel/ac_sync handshakes, form one atomic observable unit and may not be split by CompatibleP or CompatibleSys;

- R2C combinational observations are taken from the ε -quiescent fixed-point bucket and grouped according to the R2C well-formedness relation;
- same-cycle independent events may remain unordered within a unit only when no E1–E5, channel, synchronization, or witness-attribution rule orders them;
- same-cycle events that are merely coincident in one implementation are not forced to remain same-cycle in the other implementation unless they are explicitly atomic or otherwise ordered.

```
def ObservableUnitsOf
(C : ComparisonInstance)
(side : Side)
( $\tau$  : Prefix (SystemOfSide C side)) :
Set (ObservableUnit (SigmaOfSide C side)) :=
ExtractSigmaUnits
C side  $\tau$ 
(AtomicRendezvousUnits C side  $\tau$ )
(AtomicSyncUnits C side  $\tau$ )
(R2CFixedPointUnits C side  $\tau$ )
(IndependentSameCycleUnits C side  $\tau$ )
```

HappensBefore is the transitive closure of the order generators that the document requires CompatibleP / CompatibleSys to preserve:

- per-process E1 synchronization order;
- E2 signal-anchor order;
- E3 same-interface order and distinct-interface no-reverse order;
- E4 per-channel FIFO/rendezvous order and capacity-derived dependency order;
- E5 synchronization-boundary preservation;
- explicit atomic-unit membership constraints;
- same-cycle partial order recorded inside a CycleBucket when it reflects a real required dependency rather than an arbitrary linearization;
- witness-selected NB/control dependencies that affect later Σ -visible behavior, NextFront, or boundary-request attribution;
- elaboration-projection constraints when $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}(\text{E})$.

```
def HappensBefore
(C : ComparisonInstance)
(side : Side)
```



```

(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (SigmaOfSide C side)) : Prop :=
Relation.TransGen
(HappensBeforeGenerator C side τ)
u v

```

```

def HappensBeforeGenerator
(C : ComparisonInstance)
(side : Side)
(τ : Prefix (SystemOfSide C side))
(u v : ObservableUnit (SigmaOfSide C side)) : Prop :=
E1SyncOrder C side τ u v v
E2SignalAnchorOrder C side τ u v v
E3MessageOrder C side τ u v v
E4ChannelOrder C side τ u v v
E5SyncBoundaryOrder C side τ u v v
AtomicUnitOrder C side τ u v v
RequiredSameCycleBucketOrder C side τ u v v
WitnessControlOrder C side τ u v v
ElaborationProjectionOrder C side τ u v

```

CompatibleP : ComparisonInstance → Process → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleP C P τ_ref τ_post means:

- the P-local observable units of τ_ref and τ_post are related by a value-label-preserving bijection;
- explicitly atomic units are mapped to explicitly atomic units and are not split;
- the mapping preserves the P-local happens-before constraints induced by E1, E2, E3, E5, R2/R2C, and the selected witness/request-attribution facts;
- it does not require independent P-local units to occupy the same cycle bucket in both executions.

CompatibleSys : ComparisonInstance → Prefix C.Sys_ref → Prefix C.RTL_B → Prop

CompatibleSys C τ_ref τ_post means:

- $\forall P, \text{CompatibleP } C \ P \ \tau_{\text{ref}} \ \tau_{\text{post}};$

- all explicit abstract channels in the chosen Sys_ref / RTL_B comparison satisfy the applicable E4 channel semantics at the agreed abstract boundaries;
- the system-level observable-unit mapping preserves the mandated global happens-before / $\leq$ constraints and value labels;
- explicitly atomic multi-endpoint units remain atomic on both sides;
- no equality of CycleBucket lists, no equality of bucket cycle numbers, and no bucket-by-bucket equality of sigmaEvents is required.

Thus the bijection used by CompatibleSys is over ObservableUnitsOf, and its ordering obligation is preservation of HappensBefore. Independent observable units may be placed in different cycle buckets or grouped differently in Sys_ref and RTL_B, provided their Σ labels, values, atomicity requirements, and required happens-before constraints are preserved.

The notation $\tau_{\text{ref},\text{system}} \approx_{\text{sys}} \tau_{\text{post},\text{system}}$ is an abbreviation for $\text{CompatibleSys } C \tau_{\text{ref}} \tau_{\text{post}}$. It is a directed compatibility predicate, not a symmetric equivalence relation. The main construction direction is Post $\rightarrow$ Ref; the reverse direction is restricted to RTL-consistent source prefixes.

J.0:

theorem J0_pairwise_composition

(C : ComparisonInstance)

(hC : C.WellFormed)

(hB : BRules C)

(hR : RRules C)

(hE : ERules C)

(hIC : IssueCommitWF C)

(hContracts : ModelingContracts C)

(τ_{ref} : Prefix C.Sys_ref)

(τ_{post} : Prefix C.RTL_B)

(hProc : $\forall P, \text{CompatibleP } C P \tau_{\text{ref}} \tau_{\text{post}}$)

(hChan : ChannelsWellFormedAndMatched C $\tau_{\text{ref}} \tau_{\text{post}}$) :

CompatibleSys C $\tau_{\text{ref}} \tau_{\text{post}}$

J.1:

structure JAssumptions

(C : ComparisonInstance)

(W : WitnessBundle C) where
 wellformed : C.WellFormed
 bRules : BRules C
 rRules : RRules C
 eRules : ERules C
 issueCommitWF : IssueCommitWF C
 implementation : ImplementationAssumptions C
 witnessCompatible : WC C W
 contracts : ModelingContracts C
 environment : ReactiveEnvironmentWF C.env

theorem J1_post_to_ref
 (C : ComparisonInstance)
 (W : WitnessBundle C)
 (hJ : JAssumptions C W)
 (τ_{post} : Prefix C.RTL_B)
 (hExec : Exec_post C τ_{post}) :
 $\exists \tau_{\text{ref}}$,
 Exec_ref C τ_{ref} $\wedge$
 CompatibleSys C τ_{ref} τ_{post}

Restricted reverse:
 theorem J1_ref_to_post_restricted
 (C : ComparisonInstance)
 (W : WitnessBundle C)
 (hJ : JAssumptions C W)
 (τ_{ref} : Prefix C.Sys_ref)
 (hExec : Exec_ref C τ_{ref})
 (hConsistent : RTLConsistentSourcePrefix C W τ_{ref}) :
 $\exists \tau_{\text{post}}$,
 Exec_post C τ_{post} $\wedge$
 CompatibleSys C τ_{ref} τ_{post}

Cutpoint correspondence:
 structure CutpointCorrespondence
 (C : ComparisonInstance)

```

( $\sigma_{\text{ref}}$  : State)
( $\sigma_{\text{post}}$  : State) where

nextfront_agreement : Prop
frontier_guard_value_boundaryreq_agreement : Prop
buffered_channel_state_agreement : Prop
raw_rendezvous_endpoint_request_agreement : Prop
pending_blocking_enablement_agreement : Prop

direct_input_anchor_agreement : Prop
array_mapping_agreement : Prop
r2c_fixed_point_agreement : Prop

theorem J1_cutpoint_correspondence
(C : ComparisonInstance)
(W : WitnessBundle C)
(hJ : JAssumptions C W)
( $\tau_{\text{ref}}$  : Prefix C.Sys_ref)
( $\tau_{\text{post}}$  : Prefix C.RTL_B)
(hCompat : CompatibleSys C  $\tau_{\text{ref}}$   $\tau_{\text{post}}$ )
(hNorm : EpsilonNormalizedPair C  $\tau_{\text{ref}}$   $\tau_{\text{post}}$ ) :
 $\exists \sigma_{\text{ref}} \sigma_{\text{post}}$ ,
TerminalCutpoint  $\tau_{\text{ref}}$   $\sigma_{\text{ref}}$   $\wedge$ 
TerminalCutpoint  $\tau_{\text{post}}$   $\sigma_{\text{post}}$   $\wedge$ 
CutpointCorrespondence C  $\sigma_{\text{ref}}$   $\sigma_{\text{post}}$ 

```

18. Appendix K WFG and liveness

Define WFG over selected Sys_ref boundaries:

```

def WFG
(C : ComparisonInstance)
(side : Side)
( $\sigma$  : State) : Digraph Process :=
{ edge := WFGEdge C side  $\sigma$  }

```

```

def WFGEdge
(C : ComparisonInstance)
(side : Side)

```

```

( $\sigma$  : State)
(P Q : Process) : Prop :=
 $\exists$  q,
q  $\in$  Front C side P  $\sigma$   $\wedge$ 
BlockingMessageFrontierItem q  $\wedge$ 
BoundaryReq C side q  $\sigma$   $\wedge$ 
ChannelDisabled C side q  $\sigma$   $\wedge$ 
ComplementOwner C side q = Q  $\wedge$ 
InternalChannel C side q.channel

```

Edges are generated from $\text{Front_P}(\sigma)$, not arbitrary non-frontier asserted requests. This intentionally makes the WFG sparser than a graph over all asserted boundary requests: only pending blocking frontier items generate wait-for edges. Failed non-blocking polls and already-asserted non-frontier requests may affect state or occupancy through other rules, but they do not themselves create WFG dependency edges.

```

def ProcessStalledAtFront
(C : ComparisonInstance)
(side : Side)
(P : Process)
( $\sigma$  : State) : Prop :=
Front C side P  $\sigma \neq \emptyset$   $\wedge$ 
 $\forall$  x,
x  $\in$  Front C side P  $\sigma \rightarrow$ 
BlockingMessageFrontierItem x  $\rightarrow$ 
BoundaryReq C side x  $\sigma \rightarrow$ 
ChannelDisabled C side x  $\sigma$ 

```

```

def WFGClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
( $\sigma$  : State) : Prop :=
 $\forall$  P Q,
P  $\in$  D  $\rightarrow$ 
WFGEdge C side  $\sigma$  P Q  $\rightarrow$ 
Q  $\in$  D

```

```

def WFGTerminalSCC
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
( $\sigma$  : State) : Prop :=
D.Nonempty  $\wedge$ 
StronglyConnectedSubgraph (WFG C side  $\sigma$ ) D  $\wedge$ 
WFGClosed C side D  $\sigma$ 

```

```

def DrainClosed
(C : ComparisonInstance)
(side : Side)
(D : Finset Process)
( $\sigma$  : State) : Prop :=
 $\forall P,$ 
 $P \in D \rightarrow$ 
NoEnabledDrainStepLeavingD C side P D  $\sigma$ 

```

```

def ExistsClosedDrainClosedWFGCycle
(C : ComparisonInstance)
(side : Side)
( $\sigma$  : State) : Prop :=
 $\exists D,$ 
WFGTerminalSCC C side D  $\sigma$   $\wedge$ 
DrainClosed C side D  $\sigma$   $\wedge$ 
 $\forall P,$ 
 $P \in D \rightarrow$ 
ProcessStalledAtFront C side P  $\sigma$ 

```

```

def InternalMPDeadlockAt
(C : ComparisonInstance)
(side : Side)
( $\sigma$  : State) : Prop :=
EpsQuiescent  $\sigma$   $\wedge$ 
ExistsClosedDrainClosedWFGCycle C side  $\sigma$ 

```

The K.1 SCC contradiction should use this definition. The closed SCC supplies the

cycle of internal message-passing dependencies; DrainClosed rules out escape by downstream draining; ProcessStalledAtFront ensures every process in D is stalled on its current blocking frontier, not merely carrying a non-frontier asserted request.

K assumptions should preserve the document's A1–A11 as a bundle, and add Lean-side auxiliary contracts without pretending they are doc-numbered assumptions.

```
structure DockKAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
A1_point_to_point_channels : Prop
A2_chosen_Sys_ref_boundaries : Prop
A3_R_E_rule_conformance : RRules C  $\wedge$  ERules C
A4_B_rules : BRules C
A5_source_liveness : InternalMPDeadlockFree C.Sys_ref
A6_issue_commit_wf : IssueCommitWF C
A7_witness_bundle : WC C W
A8_epsilon_normalization : Prop
A9_single_transfer_per_cycle : Prop
A10_reset_empty_or_initial_accounting : Prop
A11_Kepsilon_WFG_inert : Prop
```

Lean-side additions:

```
structure KAssumptions
(C : ComparisonInstance)
(W : WitnessBundle C) where
doc : DockKAssumptions C W
contracts : ModelingContracts C
r2c? : Option (R2C_WF C)
```

This avoids fictional A12/A13 labels.

K lemmas:

```
theorem K0_frontier_blocking_classification
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
( $\sigma$  : State)
(hQ : EpsQuiescent  $\sigma$ ) :
```

...

theorem K0a_rendezvous_endpoint_request_agreement

(C : ComparisonInstance)

(W : WitnessBundle C)

(hCorr : CutpointCorrespondence C σ_{ref} σ_{post})

(c : Channel)

(hB0 : capacity c = 0) :

Req_prod c σ_{ref} = Req_prod c σ_{post} $\wedge$

Req_cons c σ_{ref} = Req_cons c σ_{post}

theorem K1_deadlock_characterization

(C : ComparisonInstance)

(W : WitnessBundle C)

(hK : KAssumptions C W)

(σ : State)

(hReach : Reachable C.RTL_B σ)

(hQ : EpsQuiescent σ) :

InternalMPDeadlockAt C .post $\sigma \leftrightarrow$

ExistsClosedDrainClosedWFGCycle C .post σ

theorem K2_dependency_refinement

(C : ComparisonInstance)

(W : WitnessBundle C)

(hK : KAssumptions C W)

(σ_{ref} σ_{post} : State)

(hCorr : CutpointCorrespondence C σ_{ref} σ_{post}) :

WFG C .ref σ_{ref} = WFG C .post σ_{post}

theorem K2a_drain_closure_transfer

(C : ComparisonInstance)

(W : WitnessBundle C)

(hK : KAssumptions C W)

(D : Finset Process)

(σ_{ref} σ_{post} : State)

(hCorr : CutpointCorrespondence C σ_{ref} σ_{post})

(hDrainPost : DrainClosed C .post D σ_{post}) :

DrainClosed C .ref D σ_{ref}

Exec-post membership at a deadlock cutpoint should be explicit:

```
theorem deadlock_cutpoint_in_Exec_post
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W)
(σ_post : State)
(hReach : Reachable C.RTL_B σ_post)
(hDead : ExistsClosedDrainClosedWFGCycle C .post σ_post)
(hIdle : C.assumptions.B.B6_idleStutterTimeDivergence) :
∃ τ_post,
TerminalCutpoint τ_post σ_post ∧
Exec_post C τ_post
```

Main K theorem:

```
theorem K1_liveness_preservation
(C : ComparisonInstance)
(W : WitnessBundle C)
(hK : KAssumptions C W) :
InternalMPDeadlockFree C.Sys_ref →
InternalMPDeadlockFree C.RTL_B
```

This is intentionally one-way. Do not state:

```
InternalMPDeadlockFree C.Sys_ref ↔ InternalMPDeadlockFree C.RTL_B
```

19. Appendix R facade

Appendix R should be formalized last as aliases/wrappers over earlier theorems.

Use Lean names that avoid the doc's R.2 collision:

```
theorem R_lemma1_front_nextfront_classification := ...
theorem R_theorem1_prefix_matching := ...
theorem R_corollary1_cutpoint_correspondence := ...
theorem R_corollary1a_rendezvous_endpoint_request_agreement := ...
theorem R_lemma2_dependency_refinement := K2_dependency_refinement
theorem R_theorem2_closed_cycle_and_one_way_drain_transfer := ...
```

Do not call cutpoint correspondence R1_cutpoint_correspondence; in the doc structure, it corresponds to a corollary, not Theorem R.1.

20. What remains as hypotheses

These remain theorem parameters or design/tool obligations:

1. R-rules R1, R2, R2C, R3, R4.
2. E-rules E1–E5.
3. B1–B6, with B2/B3 encoded temporally.
4. Implementation assumptions such as I.A0.
5. Witness compatibility WC, derived from Lemma L.2 or Corollary L.3 where applicable.
6. Direct-input contracts.
7. Array access mapping rules.
8. Elaboration package E, when $\text{Sys_ref} = \text{Sys_B}^{\text{elab}}(E)$.
9. Source-level internal message-passing deadlock freedom for K.

21. What should be proved inside Lean

Lean should prove:

- bucketed execution infrastructure;
- temporal lemmas for AlwaysFrom / EventuallyFrom;
- channel-capacity lemmas for rendezvous and buffered cases;
- issue existence/uniqueness and derived IssueTime;
- direct-input late-sampling preservation from primitive environment contracts;
- array sync-boundary commit-order preservation;
- R2C fixed-point observation lemmas from R2C_WF;
- automatic-flush frontier lemmas;
- Lemma L.2: snooping establishes WC;
- Corollary L.3: NB-CF identity witness;

- I.DR over concrete ObsSigma;
- Appendix I finite and infinite witness construction;
- Proposition J.0;
- Theorem J.1 post-to-reference;
- restricted reverse theorem for RTL-consistent source prefixes;
- Corollary J.1 cutpoint correspondence;
- K.0/K.0a/K.1/K.2/K.2a;
- deadlock cutpoint membership in Exec_post;
- Theorem K.1 one-way liveness preservation;
- Appendix R aliases.

22. Development milestones

Milestone A — Sequential, blocking-only, non-elaborated core

Scope:

- Sys_ref = Sys_B;
- sequential processes only;
- no NB calls;
- no snooping;
- no external arrays;
- no direct-input late sampling;
- no combinational R2C;
- bucketed execution model.

Targets:

J1_post_to_ref_blocking

J1_cutpoint_correspondence_blocking

K1_liveness_preservation_blocking

Milestone B — Temporal B2/B3

Add:

- infinite bucketed executions;
- AlwaysFrom, EventuallyFrom;
- concrete FairAction;
- P_push, P_pop.

Target:

K1_liveness_preservation_with_temporal_progress

Milestone C — Non-blocking under NB-CF

Add:

- Qexec;
- failed NB polls as ε ;
- successful NB transfers as Σ ;
- Corollary L.3.

Target:

J1_post_to_ref_nb_cf

Milestone D — Witness selector / snooping

Add:

- partial-prefix WitnessSelector;
- Lemma L.2.

Target:

J1_post_to_ref_with_snooping

Milestone E — Direct inputs

Add:

- DirectInputContract;
- derived lateSamplingPreservesLogicalAnchor;
- bucket placement for logical anchor vs physical late sample.

Target:

J1_post_to_ref_direct_inputs

Milestone F — External arrays

Add:

- MemoryEvent;
- ArrayAccessMapping;
- sync-relative array commit rules.

Target:

J1_post_to_ref_external_arrays

Milestone G — R2C

Add:

- combNetwork?;
- CombNetwork;
- R2C_WF;
- fixed-point bucket observation semantics.

Target:

J1_post_to_ref_with_R2C

Milestone H — Elaboration

Add:

- ElaborationPackage Sys_B Chan_outer Σ _DUT;
- Π _elab;
- $\pi_{\Sigma,E}$;
- A_E;
- explicit-channel interpretation of B, occ, BoundaryReq, ChannelEnabled, Front, and WFG.

Targets:

J1_post_to_ref_elab

K1_liveness_preservation_elab

Milestone I — Appendix R facade

Add compact R theorem wrappers after the full proof stack is available.

23. Expected difficulty

The hardest pieces remain:

1. Bucketed execution and ideal induction.
2. Temporal B2/B3 over infinite executions.
3. Concrete ObsSigma and I.DR.
4. Appendix J finite-ideal construction.
5. Direct-input late-sampling contracts.
6. External array access mapping.
7. R2C fixed-point semantics.
8. Appendix Q elaboration obligations.
9. Appendix K Exec_post membership at deadlock cutpoints.

A polished full formalization is plausibly in the **10k–18k Lean line** range if direct inputs, arrays, R2C, temporal fairness/progress, and elaboration are all included. A core sequential/blocking/non-elaborated proof would be substantially smaller.